



Module -3

CONTEXT-FREE GRAMMARS AND PUSH-DOWN AUTOMATA

overview

- ▶ CFG and CFL
- ▶ Derivations : LMD, RMD, Parse Tree
- ▶ Ambiguous Grammar .
- ▶ Push Down Automata (PDA)
- ▶ Instantaneous Description - Language of PDA:
- ▶ Acceptance by Final state, Acceptance by empty stack
- ▶ Deterministic PDA
- ▶ Equivalence of PDA and CFG
 - ▶ CFG to PDA
 - ▶ PDA to CFG

GRAMMAR

A grammar is formally represented as a 4-tuple: $G = (N, T, P, S)$,

where N is a set of non-terminal symbols,

T is a set of terminal symbols,

P is a set of production rules that define how symbols can be replaced, and

S is the start symbol from which all valid strings are generated.

$$N \cap T = \{ \}$$

Terminal symbols are the actual components of the strings, while non-terminals are variables that can be replaced by sequences of terminals or other non-terminals.

Components of a Grammar

Non-terminal Symbols (N):

These are variables or abstract symbols that can be further broken down. They are typically represented by uppercase letters.

Terminal Symbols (T):

These are the basic building blocks of the strings in a language and cannot be broken down further. They are often represented by lowercase letters.

Production Rules (P):

These are rewrite rules that specify how a non-terminal symbol can be replaced by a sequence of other symbols (terminals and non-terminals). They are the heart of a grammar's notation.

Start Symbol (S):

A special non-terminal symbol from which the generation of all strings in the language begins.

Production Rules Notation

General form:

$A \rightarrow a$, where A is a single non-terminal symbol on the left-hand side, and a is a sequence of terminal and/or non-terminal symbols on the right-hand side.

Example of Grammar and Derivation

Consider the grammar $G = (\{S, A\}, \{a, b\}, S, \{S \rightarrow aAb, aA \rightarrow aaAb, A \rightarrow \epsilon\})$:

1. **Start with the start symbol:** S .
2. **Apply a production:** $S \rightarrow aAb$. The string becomes aAb .
3. **Apply another production:** $aA \rightarrow aaAb$. The string becomes $aaAbb$.
4. **Apply the final production:** $A \rightarrow \epsilon$. The string becomes $aaabbb$.

Chomsky Hierarchy of Language

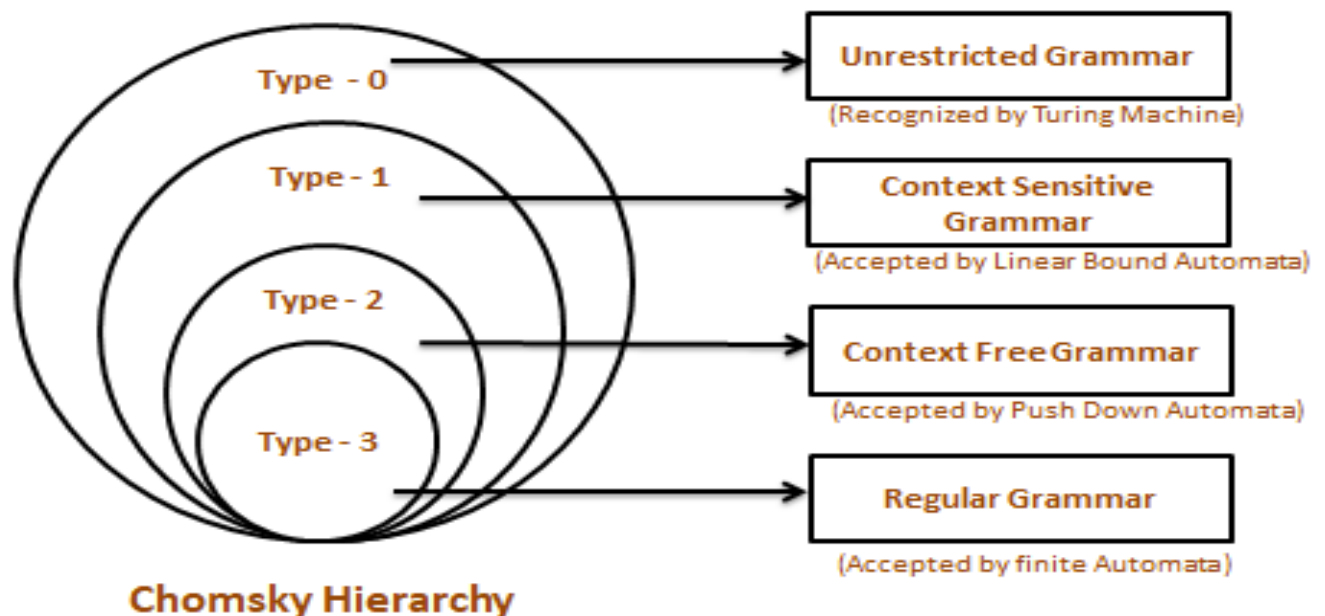
According to Chomsky Hierarchy, language is classified as follows:

Type 0 – Unrestricted grammar.

Type 1 – Context-sensitive grammar.

Type 2 – Context-free grammar.

Type 3 – Regular Grammar.



WWW.KEENOTES.COM

Chomsky Hierarchy

Type 0:

- All formal grammars fall under type-0 grammars.
- The Turing machine can detect languages with type 0 grammar.
- The Recursively Enumerable languages are another name for these languages.

Type 0 Language Production in the form of α (alpha) to β (beta) where

- **α is $(V + T)^* V (V + T)^*$**
 - V = Variables
 - T = Terminals.
- **β is $(V + T)^*$.**

In type 0 there must be at least one variable on the Left side of production.

For example:

$Sab \rightarrow ba$

$A \rightarrow S$

Here, A, S are Variables

and a, b are Terminals

Type 1:

- It is also known as Context Sensitive Language.
- It is recognized by the Linear Bound Automata.

According to the following rules, context-sensitive grammar:

- It should be Type 0
- Multiple symbols may be present on the left side of the production rules for the context-sensitive grammar.
- The number of symbols on the left side cannot be more than the number of symbols on the right side.
- The rule of the form $A \rightarrow \epsilon$ is not allowed unless A is a start symbol. It does not occur on the right-hand side of any rule.
- In type 1, Production is in the form of $|\alpha| \leq |\beta|$ Where the count of symbol in α is less than or equal to β .
- Also $\beta \in (V + T)^+$ i.e. β can not be ϵ

For Example:

$S \rightarrow AB$

$AB \rightarrow abc$

$B \rightarrow b$

Type 2:

- It should be Type 1
- It is Context-Free Grammar (CFG)
- It is recognized by a Pushdown automata
- The left-hand side of production can have only one variable and there is no restriction on β (right side)
- The production rule is of the form $|\alpha| = 1$.

For example:

$S \rightarrow AB$

$A \rightarrow a$

$B \rightarrow b$

Type 3:

- It should be Type 2
- It is a Regular Grammar
- It can be accepted by a finite-state automaton.
- It is the most restricted form of grammar.
- It can be described using regular expressions.
- These languages can be modelled by NFA or DFA.

It should be in the given form only :

$V \rightarrow VT / T$ (left-regular grammar)

(or)

$V \rightarrow TV / T$ (right-regular grammar)

For example:

$S \rightarrow a$

The above form is called strictly regular grammar.

There is another form of regular grammar called extended regular grammar. In this form:

$V \rightarrow VT^* / T^*$ (extended left-regular grammar)

(or)

$V \rightarrow T^*V / T^*$ (extended right-regular grammar)

Types of Regular Grammar

Regular grammars can be further classified into two main types –

Right Linear Grammar

In right linear grammars, the non-terminal symbol in a production rule always appears at the rightmost position. For **example** –

$$A \rightarrow xB \text{ or } A \rightarrow x$$

where 'A' and 'B' are non-terminal symbols and 'x' is a terminal symbol.

Left Linear Grammar

In left linear grammars, the non-terminal symbol resides in the leftmost position. For **example** –

$$A \rightarrow Bx \text{ or } A \rightarrow x$$

Where 'A' and 'B' are non-terminal symbols and 'x' is a terminal symbol.

Construction of a Regular Grammar from the given DFA M:

Let $M = (\{q_0, q_1, \dots, q_n\}, \Sigma, \delta, q_0, F)$. If w is in $T(M)$, then it is obtained by concatenating the labels corresponding to several transitions. The first is from q_0 , and the last terminates at some final state. So for the grammar G to be constructed, productions should correspond to transitions.

We construct G as follows

$$G = (\{A_0, A_1, A_2, \dots, A_n\}, \Sigma, P, A_0)$$

P is defined by the following rules

- (i) $A_i \rightarrow aA_j$ is included in P if $\delta(q_i, a) = q_j$ does not belong to F
- (ii) $A_i \rightarrow aA_j$ and $A_i \rightarrow a$ are included in P if $\delta(q_i, a) = q_j$ belongs to F

Construction of a Transition System M accepting $L(G)$ for a given regular grammar G :

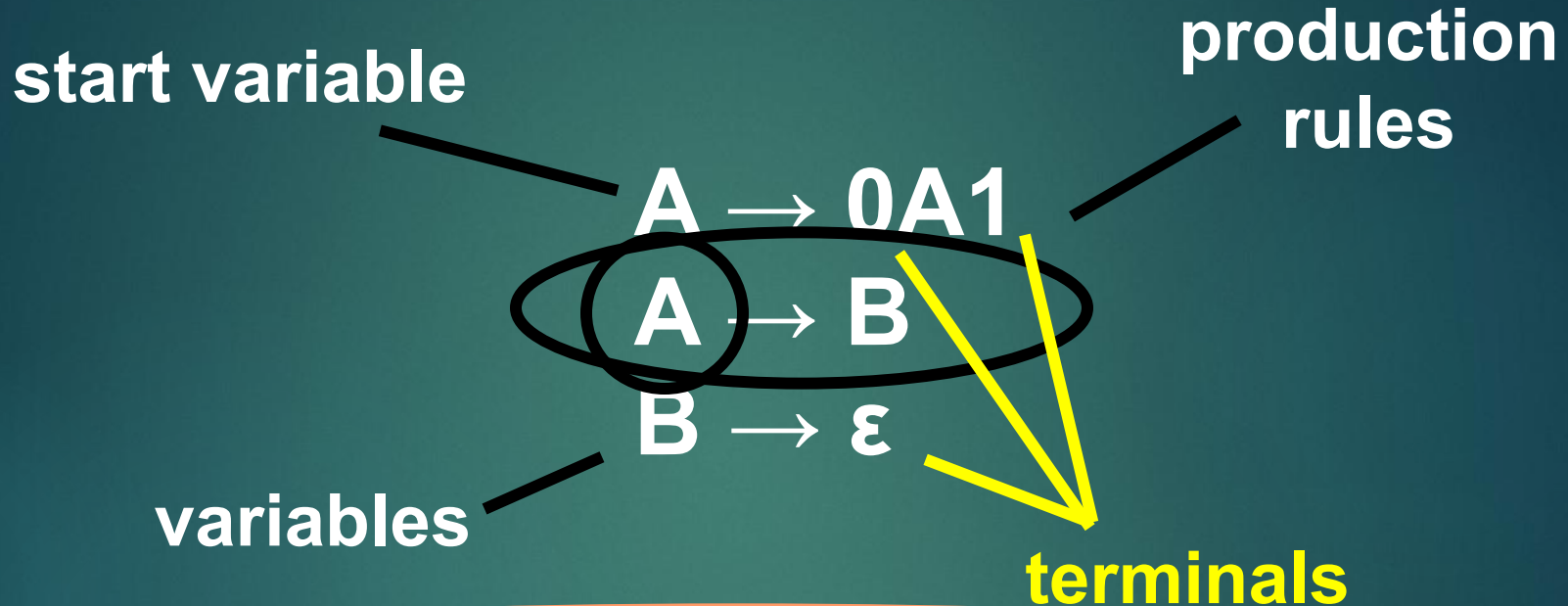
Let $G=(\{A_0,A_1,\dots,A_n\}, \Sigma, P,A_0)$. We construct the G transition system M whose

- a) States correspond to variables
- b) Initial state corresponds to A_0
- c) Transitions in M correspond to productions in P . As the last production applied in any derivation is of the form $A_i \rightarrow a$, the corresponding transition terminates at a new state and this is the unique final state.

We define $M=(\{q_0,q_1,\dots,q_n\}, \Sigma, \delta,q_0,F)$. δ is defined as follows

- i) Each production $A_i \rightarrow a A_j$ induces a transition from q_i to q_j with label a .
- ii) Each production $A_k \rightarrow a$ induces a transition from q_k to q_f with label a .

CONTEXT-FREE GRAMMARS



$A \Rightarrow 0A1 \Rightarrow 00A11 \Rightarrow 00B11 \Rightarrow 00 \epsilon 11$

$\Rightarrow$ (yields) Derivation

$A \Rightarrow^* 0011$
(derives)

Non-deterministic

We say: $00 \epsilon 11$ is
generated by the
Grammar

CONTEXT-FREE GRAMMARS

$A \rightarrow 0A1$

$A \rightarrow B$

$B \rightarrow \epsilon$

(OR)

$A \rightarrow 0A1 \mid B$

$B \rightarrow \epsilon$

(OR)

$A \rightarrow 0A1 \mid \epsilon$

CONTEXT-FREE LANGUAGES

A context-free grammar (CFG) is a tuple $G = (V, T, P, S)$, where:

V is a finite set of variables

T is a finite set of terminals (disjoint from V)

P is set of production rules of the form $A \rightarrow \alpha$, where $A \in V$ and $\alpha \in (V \cup T)^*$

$S \in V$ is the start variable

$$G = \{ \{S\}, \{0,1\}, P, S \} \quad P = \{ S \rightarrow 0S1, S \rightarrow \varepsilon \}$$

$$L(G) =$$

CONTEXT-FREE GRAMMARS

A context-free grammar (CFG) is a tuple $G = (V, T, P, S)$, where:

V is a finite set of variables

T is a finite set of terminals (disjoint from V)

P is set of production rules of the form $A \rightarrow \alpha$, where $A \in V$ and $\alpha \in (V \cup T)^*$

$S \in V$ is the start variable

$L(G) = \{w \in T^* \mid S \Rightarrow^* w\}$ Strings Generated by G

CONTEXT-FREE LANGUAGES

A context-free grammar (CFG) is a tuple $G = (V, T, P, S)$, where:

V is a finite set of variables

T is a finite set of terminals (disjoint from V)

P is set of production rules of the form $A \rightarrow \alpha$, where $A \in V$ and $\alpha \in (V \cup T)^*$

$S \in V$ is the start variable

$G = \{ \{S\}, \{0,1\}, P, S \} \quad P = \{ S \rightarrow 0S1, S \rightarrow \epsilon \}$

$L(G) = \{ 0^n 1^n \mid n \geq 0 \}$ Strings Generated by G

1. WRITE A CFG FOR EVEN-LENGTH PALINDROMES

$$S \rightarrow \sigma S \sigma \text{ for all } \sigma \in T$$

$$S \rightarrow \varepsilon$$

(OR)

If $T = \{a, b\}$

$$S \rightarrow aSa \mid bSb \mid \varepsilon$$

$W = \{\varepsilon, aa, bb, abba, baab, aaaa, bbbb, \dots\}$

2.WRITE A CFG FOR Odd-LENGTH PALINDROMES

$S \rightarrow \sigma S \sigma$ for all $\sigma \in T$

$S \rightarrow \sigma$

(OR)

If $T=\{a,b\}$

$S \rightarrow aSa \mid bSb \mid a \mid b$

$W=\{a, b, aba, bab, abbba, babab, aabaa, bbabb, \dots\}$

3.WRITE A CFG FOR PALINDROMES

$S \rightarrow \sigma S \sigma$ for all $\sigma \in T$

$S \rightarrow \sigma \mid \epsilon$

(OR)

If $T=\{a,b\}$

$S \rightarrow aSa \mid bSb \mid a \mid b \mid \epsilon$

$W=\{\epsilon, a, b, aa, bb, aba, bab, abba, baab, aaaa, bbbb, abbba, babab, \dots\}$

4.WRITING A CFG FOR THE EMPTY SET

$$G = \{ \{S\}, T, \emptyset, S \}$$

5.WRITING A CFG FOR $L=\{ba\}$

$$G = \{ \{S,B\}, T, S \rightarrow Ba, B \rightarrow b, S \}$$

(OR)

$$G = \{ \{S,A\}, T, S \rightarrow bA, A \rightarrow a, S \}$$

1. WRITE A CFG FOR $L = \{a^n \mid n \text{ is odd}\}$

$$G = \{ \{S\}, \{a\}, S \rightarrow a \mid aaS, S \}$$

2. WRITE A CFG FOR $L = \{a^n \mid n \text{ is even}\}$

$$G = \{ \{S\}, \{a\}, S \rightarrow \epsilon \mid aaS, S \}$$

3. WRITE A CFG FOR $L = \{wcw^R$

$$G = \{ \{S\}, \{a, b, c\}, \{w \in (a+b)^*\} \mid S \rightarrow c \mid aSa \mid bSb, S \}$$

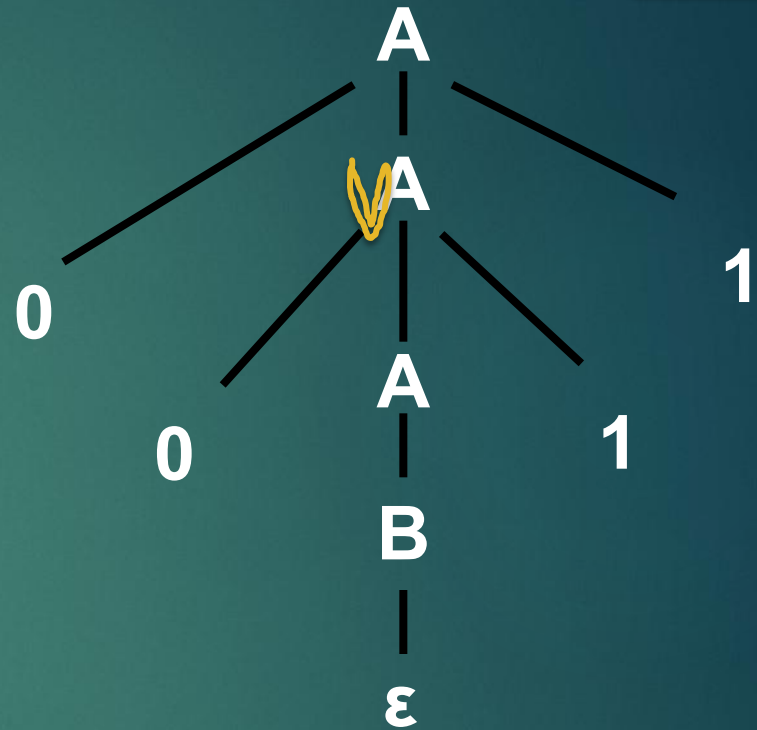
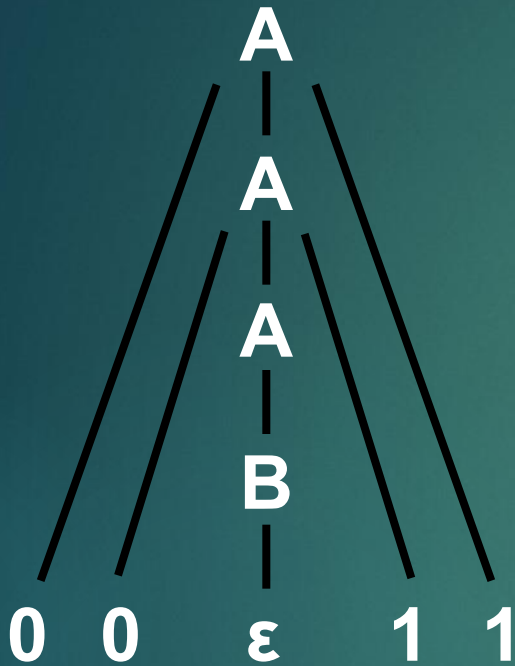
4. CFG for $L = \{\text{equal number of } a\text{'s and } b\text{'s}\}$

$$G = \{ \{S, A, B\}, \{a, b\}, S \rightarrow aB \mid bA \mid \epsilon, \\ B \rightarrow bS \mid aBB \mid b, A \rightarrow aS \mid bAA \mid a, S \}$$

Derivation Tree or Parse Tree:

- The derivation of a CFG can be represented using trees called derivation trees.
- A derivation tree for a CFG $G=(V,T,P,S)$ is a tree satisfying the following
 1. every vertex has a label which is a variable or terminal or ϵ .
 2. The root has label S
 3. The label of an internal vertex is a variable.
 4. If the vertices $N_1, N_2, \dots, N_k$ written with labels $X_1, X_2, \dots, X_k$ are the sons of vertex “N” with label A then $A \rightarrow X_1, X_2, \dots, X_k$ is a production in P.
 5. A vertex n is a leaf if its label is ‘a’ belongs to T or ϵ

PARSE TREES



$A \Rightarrow 0A1 \Rightarrow 00A11 \Rightarrow 00B11 \Rightarrow 00 \epsilon 11$



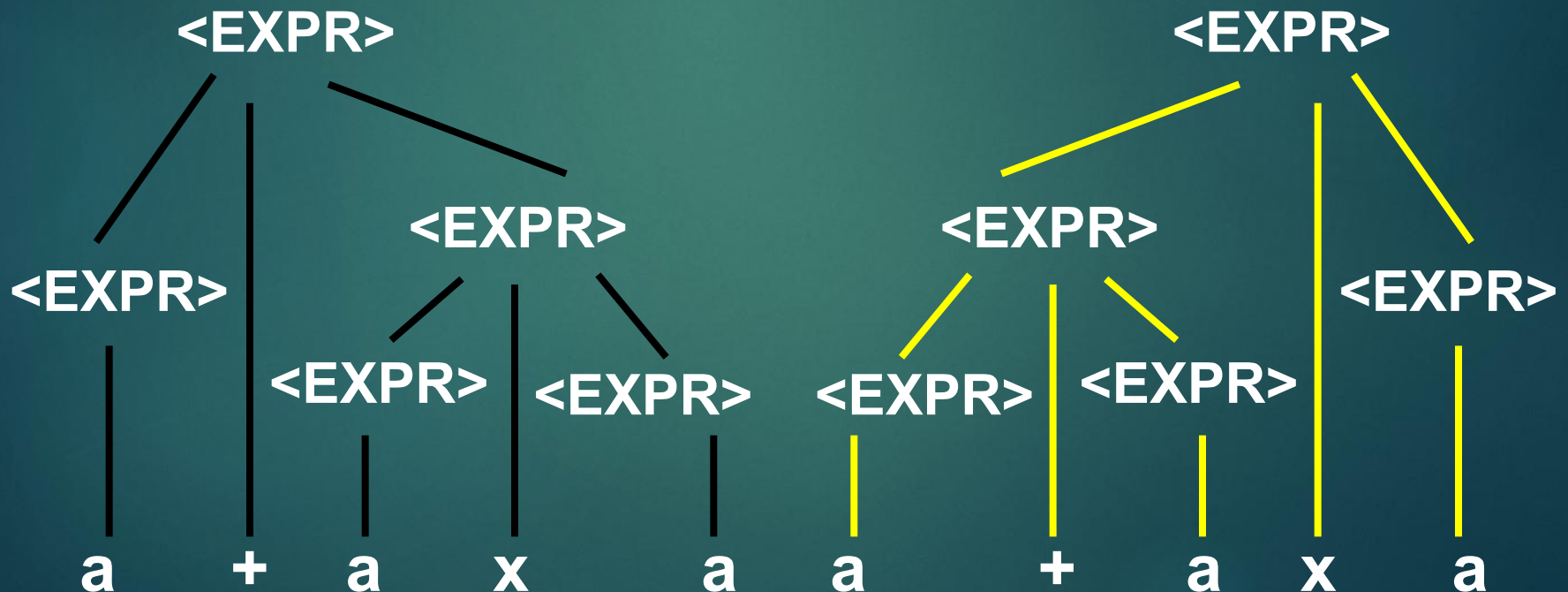
$\langle \text{EXPR} \rangle \rightarrow \langle \text{EXPR} \rangle + \langle \text{EXPR} \rangle$

$\langle \text{EXPR} \rangle \rightarrow \langle \text{EXPR} \rangle \times \langle \text{EXPR} \rangle$

$\langle \text{EXPR} \rangle \rightarrow (\langle \text{EXPR} \rangle)$

$\langle \text{EXPR} \rangle \rightarrow a$

Build a parse tree for $a + a \times a$



$G = (\{S, A\}, \{a, b\}, P, S)$ where P consists of

$S \rightarrow aAS / a / SS$

$A \rightarrow SbA / ba$

select string aabaa



$G = (\{S, A\}, \{a, b\}, P, S)$ where P consists of

$S \rightarrow aAS / a / SS$

$A \rightarrow SbA / ba$

select string aabaa

Derivations

To infer whether the given input string belongs to the given CFG, we can have two approaches:

Using the rules from head to body

- ▶ Two types of Derivations:

- ▶ Left Most Derivation(LMD): If at each step in a derivation, a production is applied to the leftmost variable, then it is called leftmost derivation.
- ▶ Right Most Derivation(RMD): - production is applied to the rightmost variable.

Left Most Derivation

- ▶ $G=(V,T,P,S)$ $V=\{E\}$
- ▶ $T=\{+,* ,id\}$ $P=\{E \rightarrow E+E \mid E * E \mid id \}$

Construct the leftmost derivation for $id+id*id$

$E \xRightarrow{lm} E+E$
 $\xRightarrow{lm} id +E$
 $\xRightarrow{lm} id +E * E$
 $\xRightarrow{lm} id +id * E$
 $\xRightarrow{lm} id +id * id$

Right most derivation for $id+id*id$

$$\overset{rm}{E} \Rightarrow E+E$$

$$\overset{rm}{\Rightarrow} E+E*E$$

$$\overset{rm}{\Rightarrow} E+E*id$$

$$\overset{rm}{\Rightarrow} E +id*id$$

$$\overset{rm}{\Rightarrow} id +id*id$$

Consider G whose productions are $S \rightarrow aAS/a$, $A \rightarrow SbA/SS/ba$. Construct LMD and RMD trees whose yield is aabbbaa.

LMD:

$S \rightarrow aAS$
 $\rightarrow aSbAS$ [Applied $A \rightarrow SbA$]
 $\rightarrow aabAS$ [$S \rightarrow a$]
 $\rightarrow aabbaS$ [$A \rightarrow ba$]
 $\rightarrow aabbbaa$ [$S \rightarrow a$]

RMD:

$S \rightarrow aAS \rightarrow aSa \rightarrow aSbAa \rightarrow aSbbbaa \rightarrow aabbbaa$

Sentential Form

- ▶ The strings produced /derived from the start symbol is called 'sentential form'. If $G=(V,T,P,S)$ is a CFG, then α in $(V \cup T)^*$ such that
- ▶ $S \xRightarrow{*} \alpha$ is a sentential form
- ▶ If $S \xRightarrow{*}_{lm} \alpha$ then α is left sentential form
- ▶ If $S \xRightarrow{*} \alpha$ then α is right sentential form^m

The yield of a Parse Tree

- The string obtained by concatenating the leaves of a parse tree from the left is called the yield of a parse tree.

$$S \rightarrow A1B$$

$$A \rightarrow 0A \mid \varepsilon$$

$$B \rightarrow 0B \mid 1B \mid \varepsilon$$

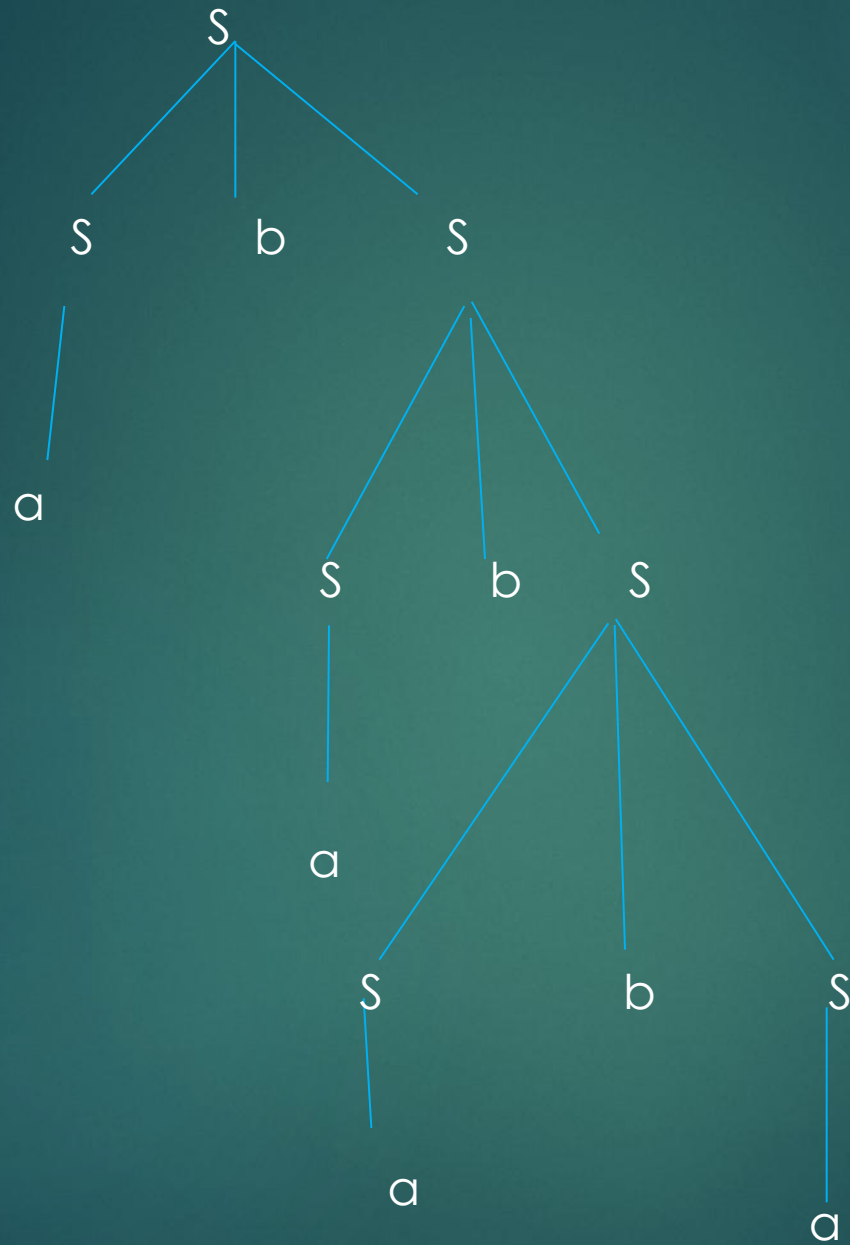
Draw the Parse tree for the yields 1001, 00101

Ambiguous Grammar

- ▶ A CFG $G=(V,T,P,S)$ is ambiguous if there is at least one string w in T^* is having two LMD trees or two RMD trees , each with the same root S and some yield w .
- ▶ If G is a grammar $S \rightarrow SbS/a$. Show that G is ambiguous .

Consider $w=abababab$ in $L(G)$. We get two LMD or RMD derivation trees for w . thus G is ambiguous.

LMD 1:



grammar $S \rightarrow SbS / a$

$G = (\{S\}, \{a, b, +, *\}, P, S)$ where P consists of $S \rightarrow S + S / S * S / a / b$. show that G is ambiguous.

Simplification of CFG or Elimination of Useless Symbols:

If G is a CFG such that $L(G) \neq \emptyset$, we can find an equivalent grammar G' , such that each variable in G' derives some terminal string.

1. Construction of V' :

We define $w_i \subseteq V$ by recursion.

$w_i = \{ A \in V \mid \text{there exists a production } A \rightarrow w \text{ where } w \in T^* \}$

$w_{i+1} = w_i \cup \{ A \in V \mid \text{there exists some productions } A \rightarrow \alpha \text{ with } \alpha \in (T \cup w_i)^* \}$

2. Construction of P' :

$P' = \{ A \rightarrow \alpha \text{ where } A, \alpha \in (V' \cup T)^* \}$

3. We can define $G' = (V' \cup T, P', S)$ S is in V'

Elimination of Useless Production Rules:

For every CFG $G=(V,T,P,S)$.we can construct an equivalent grammar $G' = (V',T', P',S)$ such that every symbol in $(V' \cup T')$. Appears in some sentential form.

1. Construction of w_i for $i \geq 1$:

(i) $w_1=\{S\}$

(ii) $w_{i+1} = w_i \cup \{ x \in (V \cup T) \text{ there exists a production}$

$A \rightarrow a \text{ with } A \in w_i \text{ and } a \text{ containing the symbol } x\}$

$w_i = w_{i+1} \text{ for some } i$

2. Construction of V' , T' and P' :

$$V' = V \cap w_i \qquad T' = T \cap w_i$$

$$P' = \{ A \rightarrow a / A \in w_i \}$$

Normal Forms

Normal forms are standardized ways of representing grammars in a simplified or structured form.

Chomsky Normal Form (CNF):

A **context-free grammar (CFG)** is in **Chomsky Normal Form** if every rule follows one of these forms:

1. $A \rightarrow BC$ (where A , B , and C are non-terminals, B and C are not the start symbols)
2. $A \rightarrow a$ (where A is a non-terminal, and " a " is a terminal)
3. Start symbol generating ϵ . (e.g.; $S \rightarrow \epsilon$)



For example:

1. $G1 = \{S \rightarrow AB, S \rightarrow c, A \rightarrow a, B \rightarrow b\}$

2. $G2 = \{S \rightarrow aA, A \rightarrow a, B \rightarrow c\}$

The production rules of Grammar $G1$ satisfy the rules specified for CNF, so the grammar $G1$ is in CNF.

However, the production rule of Grammar $G2$ does not satisfy the rules specified for CNF as $S \rightarrow aZ$ contains terminal followed by non-terminal. So the grammar $G2$ is not in CNF.

Algorithm to Convert a CFG into Chomsky Normal Form –

Step 1 – Remove Null productions. (Using the Null production removal algorithm eliminates Null productions)

Step 2 – Remove unit productions. (Using the Unit production removal algorithm)

Step 3 – Replace each production $A \rightarrow B_1 \dots B_n$ where $n > 2$ with $A \rightarrow B_1 C$ where $C \rightarrow B_2 \dots B_n$. Repeat this step for all productions having two or more symbols in the right side.

For example, $S \rightarrow ASB$ can be decomposed as:

1. $S \rightarrow RS$
2. $R \rightarrow AS$

Step 4 – If the right side of any production is in the form $\mathbf{A} \rightarrow \mathbf{aB}$ where $\mathbf{a}$ is a terminal and $\mathbf{A}, \mathbf{B}$ are non-terminal, then the production is replaced by $\mathbf{A} \rightarrow \mathbf{XB}$ and $\mathbf{X} \rightarrow \mathbf{a}$. Repeat this step for every production which is in the form $\mathbf{A} \rightarrow \mathbf{aB}$.

For example, production $S \rightarrow aA$ can be decomposed as:

1. $S \rightarrow RA$

2. $R \rightarrow a$

Step 1: Remove Null productions:

A Nullable variable in a CFG $G=(V,T,P,S)$ is defined as follows

1. Any variable A for which P contains the production $A \rightarrow \varepsilon$ is Nullable.
2. If P contains the production $A \rightarrow B_1 B_2 \dots B_n$ and $B_1 B_2 \dots B_n$ are Nullable variables, then A is Nullable.
3. No other variables in V are nullable.

Algorithm : Finding an equivalent CFG with no ϵ productions):

Given a CFG $G=(V,T,P,S)$ construct a CFG $G1=(V,T,P1,S)$ with no ϵ productions.

1. Initialize $P1=P$.
2. Find all nullable variables in V .
3. For every production, $A \rightarrow a$ in P , add to $P1$ every production that can be obtained from this one by deleting from a one or more occurrences of Nullable variables in a .
4. Delete all ϵ productions in $P1$. also delete any duplicates, as well as productions of the form $A \rightarrow A$.

Step 2 : Remove unit productions:

1. If $A \rightarrow B$ is a production, B is A -derivable
2. If C is A -derivable, $C \rightarrow B$ is a production and $B \neq A$ then B is A -derivable

Algorithm : Finding an equivalent CFG with no unit productions

Given a CFG $G=(V,T,P,S)$ with no ϵ productions construct a grammar $G_1=(V,T,P_1,S)$ having no unit productions as follows:

1. Initialize $P_1=P$.
2. For each $A \in V$ find the set of A derivable variables.
3. For every pair (A,B) such that B is A -derivable, and every non unit production $B \rightarrow a$, add the production $A \rightarrow a$ to P_1 , if it is not already in P_1 .
4. Delete all unit productions from P_1 .

Step 3 and 4: Restricting the right sides of productions to single terminal or string of two variable.

Example: Converting a CFG to CNF

Let us take an example to convert CFG to CNF. Consider the given grammar G1:

$$S \rightarrow ASB$$

$$A \rightarrow aAS/a/\epsilon$$

$$B \rightarrow SbS/A/bb$$

Greibach Normal Form (GNF):

A context-free grammar (CFG) is in Greibach Normal Form if every rule follows this form:

1. $A \rightarrow a\alpha$ (where "a" is a terminal and " α " is a sequence of non-terminals or empty)

Lemma 1:

Define an A- production to be a production with variable A on the left. Let $G=(V,T,P,S)$ be a CFG. Let $A \rightarrow \alpha_1 B \alpha_2$ be a production in P and $B \rightarrow \beta_1 / \beta_2 / \dots / \beta_r$ be the set of all B-productions. Let $G_1=(V,T,P_1,S)$ be obtained from G by deleting the production .

$A \rightarrow \alpha_1 B \alpha_2$ from P and adding the productions

$A \rightarrow \alpha_1 \beta_1 \alpha_2 / \alpha_1 \beta_2 \alpha_2 / \dots / \alpha_1 \beta_r \alpha_2$

then $L(G)=L(G_1)$

Definition: A production $U \in R$ is said to be in the form **left recursion**, if $U : A \rightarrow A\alpha$ for some $A \in V$.

Left recursion can be eliminated by the following Lemma 2:

Let $G=(V,T,P,S)$ be a CFG. Let $A \rightarrow A\alpha_1/A\alpha_2/.../A\alpha_r$ be the set of A productions for which A is the leftmost symbol of the right-hand side.

$A \rightarrow \beta_1 / \beta_2 // \beta_s$ be the remaining A-productions.

Let $G_1=(V \cup \{B\}, T, P_1, S)$ be the CFG formed by adding the variable B to V and replacing all the A-Productions by the productions.

$A \rightarrow \beta_i$

$A \rightarrow \beta_i B$

Where $1 \leq i \leq s$

$B \rightarrow \alpha_i$

$B \rightarrow \alpha_i B$

where $1 \leq i \leq r$

The stepwise algorithm is as follows:

1. Eliminate null productions, unit productions and useless symbols from the grammar G and then construct a $G' = (V', T, R', S)$ in **Chomsky Normal Form (CNF)** generating the language $L(G') = L(G) - \{\epsilon\}$.
2. Rename the variables like $A_1, A_2, \dots, A_n$ starting with $S = A_1$.
3. Modify the rules in R' so that if $A_i \rightarrow A_j \gamma \in R'$ then $j > i$
4. Starting with A_1 and proceeding to A_n this is done as follows:
 - (a) Assume that productions have been modified so that for $1 \leq i \leq k$, $A_i \rightarrow A_j \gamma \in R'$ only if $j > i$
 - (b) If $A_k \rightarrow A_j \gamma$ is a production with $j < k$, generate a new set of productions substituting for the A_j the body of each A_j production.
 - (c) Repeating (b) at most $k - 1$ times we obtain rules of the form $A_k \rightarrow A_p \gamma, p \geq k$
 - (d) Replace rules $A_k \rightarrow A_k \gamma$ by removing left-recursion as stated above.
5. Modify the $A_i \rightarrow A_j \gamma$ for $i = n - 1, n - 2, \dots, 1$ in desired form at the same time change the Z production rules.

1. Convert the following CFG to CNF (Chomsky Normal Form)

$$S \rightarrow ASB$$

$$A \rightarrow aAS / a / \epsilon$$

$$B \rightarrow SbS / A/bb$$

2. Convert the following CFG to GNF (Greibach Normal Form)

$$S \rightarrow AB|BC$$

$$A \rightarrow AB|a$$

$$B \rightarrow AA|CB|b$$

$$C \rightarrow a|b$$

3. Convert the following CFG to GNF (Greibach Normal Form)

$S \rightarrow AB$

$A \rightarrow BS|b$

$B \rightarrow SA|a$

4. Consider the grammar $G = (V, T, S, P)$ with $V = \{S, A, B, C\}$, $T = \{a, b\}$, $S = S$ and the productions P defined by:

$S \rightarrow AB \mid BC$

$A \rightarrow BA \mid a$

$B \rightarrow CC \mid b$

$C \rightarrow AB \mid a$

Use the CYK algorithm to establish the membership or non-membership of $w = ababa$ in $L(G)$

Pumping Lemma (For Context Free Languages)

Pumping Lemma (for CFL) is used to prove that a language is NOT Context Free

Context Free Language

In formal language theory, a Context Free Language is a language generated by some Context Free Grammar.

The set of all CFL is identical to the set of languages accepted by Pushdown Automata.

Context Free Grammar is defined by 4 tuples as $G = \{ V, \Sigma, S, P \}$ where

V = Set of Variables or Non-Terminal Symbols

Σ = Set of Terminal Symbols

S = Start Symbol

P = Production Rule

Context Free Grammar has Production Rule of the form

$$A \rightarrow \alpha$$

where, $\alpha \in \{V \cup \Sigma\}^*$ and $A \in V$

Pumping Lemma (For Context Free Languages)

Pumping Lemma (for CFL) is used to prove that a language is NOT Context Free

If A is a Context Free Language, then, A has a Pumping Length ' P ' such that any string ' S ', where $|S| \geq P$ may be divided into 5 pieces $S = uvxyz$ such that the following conditions must be true:

- (1) $u v^i x y^i z$ is in A for every $i \geq 0$
- (2) $|v y| > 0$
- (3) $|v x y| \leq P$

To prove that a Language is Not Context Free using Pumping Lemma (for CFL) follow the steps given below: (We prove using CONTRADICTION)

- > Assume that A is Context Free
- > It has to have a Pumping Length (say P)
- > All strings longer than P can be pumped $|S| \geq P$
- > Now find a string 'S' in A such that $|S| \geq P$
- > Divide S into uvxyz
- > Show that $u v^i x y^i z \notin A$ for some i
- > Then consider the ways that S can be divided into uvxyz
- > Show that none of these can satisfy all the 3 pumping conditions at the same time
- > S cannot be pumped == CONTRADICTION

Pumping Lemma (for Context Free Languages) - Example (Part-1)

Show that $L = \{ a^N b^N c^N \mid N \geq 0 \}$ is Not Context Free

-> Assume that L is Context Free

-> L must have a pumping length (say P)

-> Now we take a string S such that $S = a^P b^P c^P$

-> We divide S into parts $u v x y z$

Eg. $P = 4$ So, $S = a^4 b^4 c^4$

Case I: v and y each contain only one type of symbol

$\underbrace{a a a a}_{u v} \underbrace{b b b b}_x \underbrace{c c c c}_{y z}$

$u v^i x y^j z$ $(i = j)$
 $u v^2 x y^2 z$

$a a a a a b b b b c c c c c$

$a^6 b^4 c^5$
↓

Case II: Either v or y has more than one kind of symbols

$\underbrace{aaaa}_{u} \underbrace{bbbb}_{v} \underbrace{bb}_{xy} \underbrace{cccc}_{z}$

$uv^i xy^i z \quad (i=2)$

$uv^2 xy^2 z$

$\underline{a^N b^N c^N}$

$aa aabb aabbbbbb cccc \notin L$

Pumping Lemma (for Context Free Languages) - Example

Show that $L = \{ ww \mid w \in \{0,1\}^* \}$ is NOT Context Free

-> Assume that L is Context Free

-> L must have a pumping length (say P)

-> Now we take a string S such that $S = 0^P 1^P 0^P 1^P$

-> We divide S into parts $u v x y z$

Pumping Lemma (For Context Free Languages)

Pumping Lemma (for CFL) is used to prove that a language is NOT Context Free

If A is a Context Free Language, then, A has a Pumping Length ' P ' such that any string ' S ', where $|S| \geq P$ may be divided into 5 pieces $S = uvxyz$ such that the following conditions must be true:

(1) $u v^i x y^i z$ is in A for every $i \geq 0$

(2) $|v y| > 0$

(3) $|v x y| \leq P$

Eg. $P = 5$ So, $S = 0^5 1^5 0^5 1^5$

Case 1: vxy does not straddle a boundary

00000 11111 00000 11111
 $u \quad vxy \quad z$

000001111110000011111

$\underbrace{0^5 1^7}_{\neq} \underbrace{0^5 1^5} \notin L$

Case 2a: vxy straddles the first boundary

00000 11111 00000 11111
 $u \quad v \quad xy \quad z$

00000001111110000011111

$\underbrace{0^7 1^7}_{\neq} \underbrace{0^5 1^5} \notin L$

$uv^i xy^i z$

$uv^2 xy^2 z$



$uv^i xy^i z$

$uv^2 xy^2 z$

$$u v^2 x y^2 z$$
$$\underbrace{0^5 1^5} \neq \underbrace{0^7 1^7} \notin L$$

00000'11111'00000'11111
u v x y z

$$uv^2xy^2z$$
$$\underbrace{0517}_{\neq} \underbrace{0718} \notin L$$

Introduction to Pushdown Automata (PDA)

- PDA as an extension of Finite Automata (FA) with a stack
- Importance of PDAs in recognizing Context-Free Languages (CFLs)
- Real-world applications: compiler design, parsing, and formal language theory

A Pushdown Automata (PDA) is a way to implement a Context Free Grammar in a similar way we design Finite Automata for Regular Grammar.

Pushdown automata are like Nondeterministic Finite Automata but have an extra component called **Stack**.

Properties of Pushdown Automata

1. It is more powerful than Finite State Machine (FSA).
1. FSA has a very limited memory but PDA has more memory.
1. PDA = Finite State Machine + Stack.

Stack

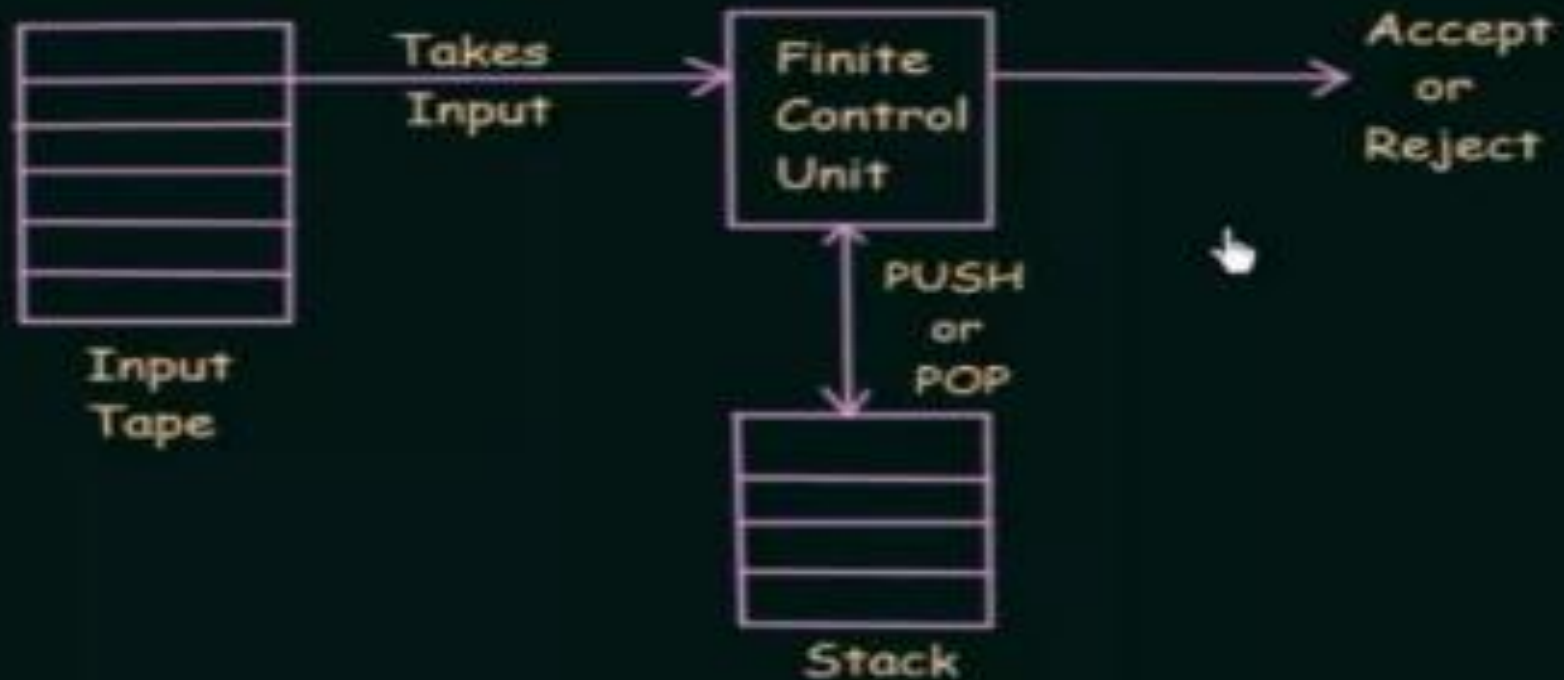
A stack is way we arrange elements one on top of another.

A stack does two basic operation:

1. PUSH: A new element is added at the top of stack.
2. POP: The top element of the stack is read and removed.

Components of Pushdown Automata

1. An Input tape.
2. A Finite Control Unit.
3. A Stack with infinite size.

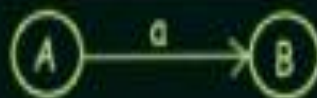


Formal Definition of PDA

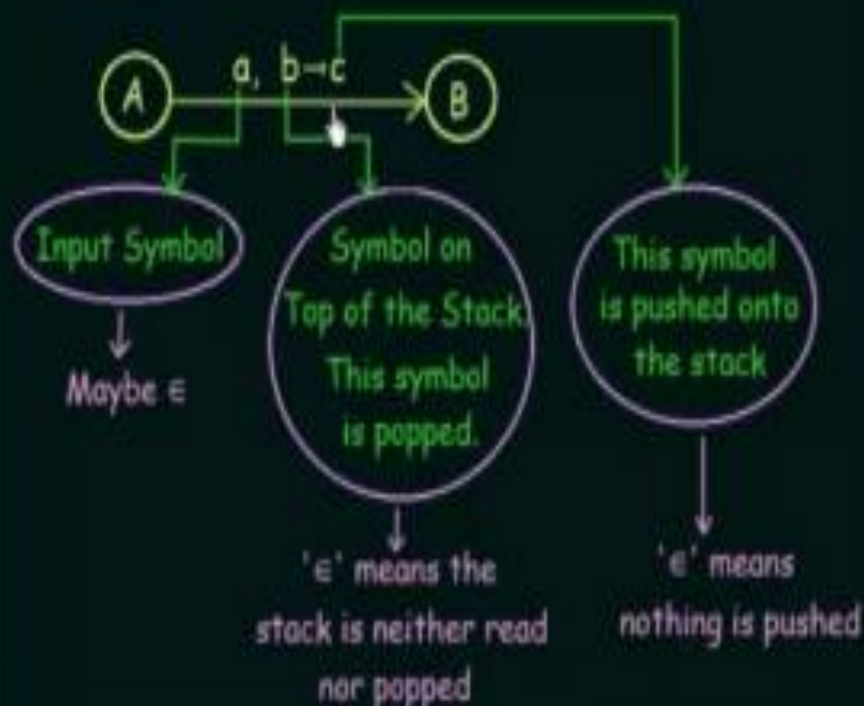
- Formal Definition: A PDA is a 7-tuple: $(Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$
 - Q : Finite set of states
 - Σ : Input alphabet (finite set of input symbols)
 - Γ : Stack alphabet (finite set of stack symbols)
 - δ : Transition function $(Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma \rightarrow P(Q \times \Gamma^*))$
 - q_0 : Start state ($q_0 \in Q$)
 - Z_0 : Initial stack symbol ($Z_0 \in \Gamma$)
 - F : Set of accepting states ($F \subseteq Q$)
- Detailed explanation of each component with examples
 - Example: $Q = \{q_1, q_2\}$, $\Sigma = \{a, b\}$, $\Gamma = \{\$, A\}$
 - $q_0 = q_1$, $Z_0 = \$$, $F = \{q_2\}$

Graphical Notation (PDA)

Finite State Machine



Pushdown Automata

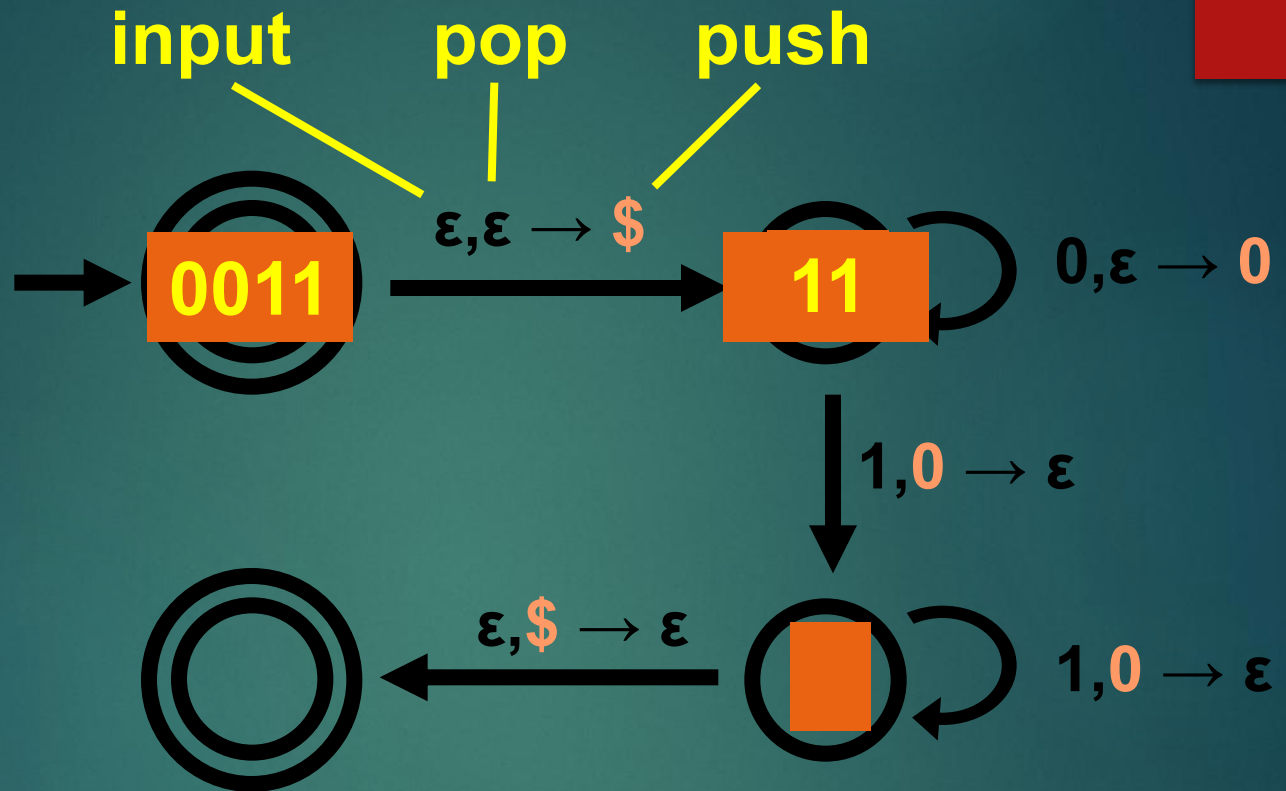


PDA Transition Function

- The transition function $\delta: (Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma \rightarrow P(Q \times \Gamma^*))$ is the core of a PDA
- $\delta(\text{current state, input symbol, top of stack}) = (\text{new state, stack operation})$
- Types of Transitions:
 - Reading Input: Consuming an input symbol and changing state/stack
 - Stack Manipulation: Pushing or popping symbols on the stack
 - ϵ -Transitions: Changing state without consuming input
- Example:
 - $\delta(q_1, a, \$) = \{(q_1, A\$)\}$ // Read 'a', push 'A' onto the stack
 - $\delta(q_1, b, A) = \{(q_2, \epsilon)\}$ // Read 'b', pop 'A' from the stack
 - $\delta(q_2, \epsilon, \$) = \{(q_2, \$)\}$ // ϵ -transition, remain in q_2

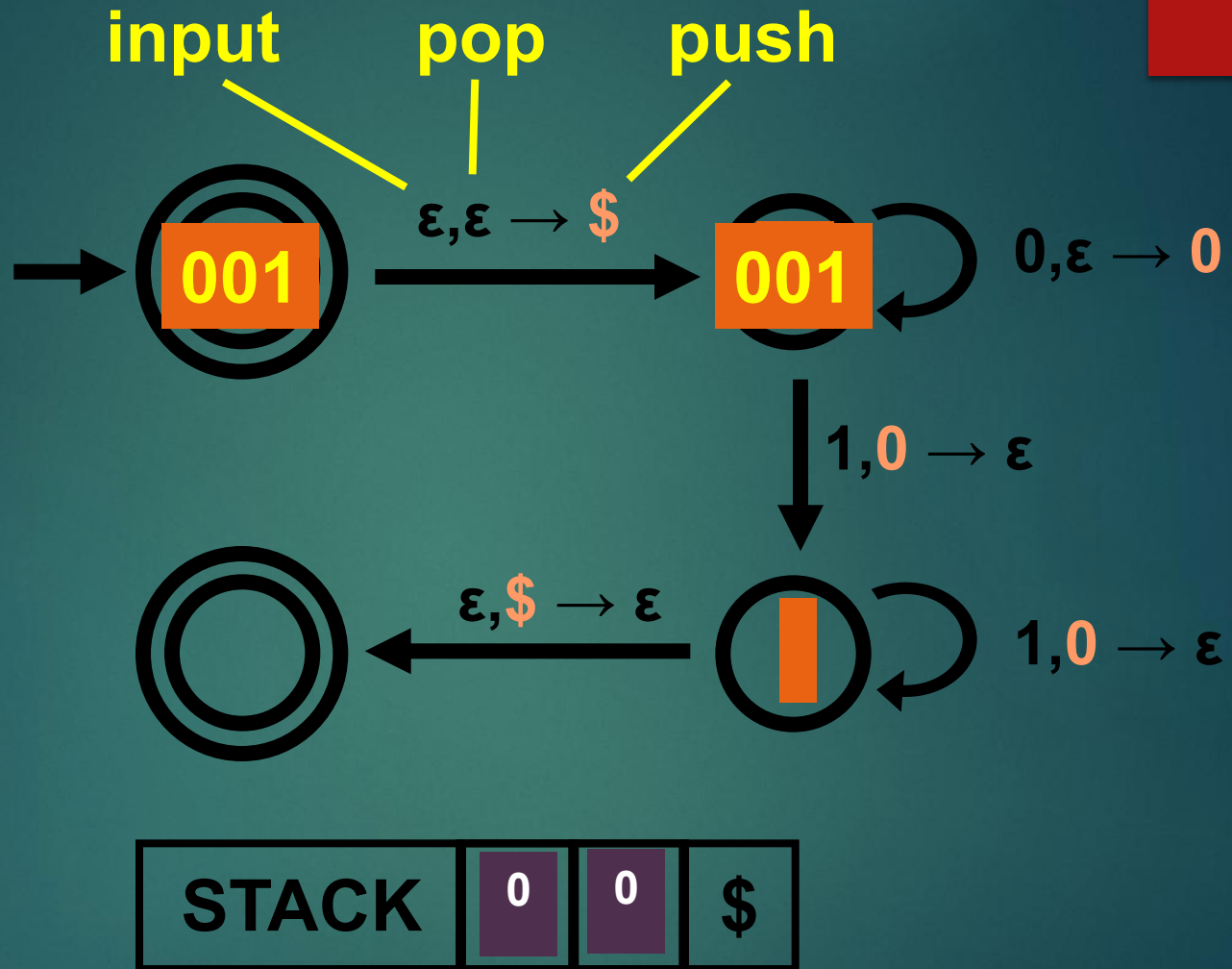
Language Acceptance by PDA

- **Acceptance by Final State:** The PDA accepts the input string if it ends in an accepting state.
- **Acceptance by Empty Stack:** The PDA accepts the input string if the stack is empty after processing the input.
- **Equivalence:** Acceptance by final state and acceptance by empty stack are equivalent.
- **Example:**
 - Language $L = \{a^n b^n \mid n \geq 0\}$
 - PDA transitions to accept L by empty stack
 - Trace the PDA with input "aabb" to show acceptance



STACK

Non-deterministic



PDA that recognizes $L = \{ 0^n 1^n \mid n \geq 0 \}$

Definition: A (*non-deterministic*) PDA is a 7 tuple machine $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$, where:

Q is a finite set of states

Σ is the input alphabet

Γ is the stack alphabet

$\delta : Q \times (\Sigma \cup \epsilon) \times \Gamma \rightarrow Q \times \Gamma^*$

$q_0 \in Q$ is the start state

$Z_0 \in \Gamma$ The start symbol of the stack

$F \subseteq Q$ is the set of accept states

Γ^* is the string of stack symbols

Let $w \in \Sigma^*$ and suppose w can be written as $w_1 \dots w_n$ where $w_i \in \Sigma_\epsilon$ (recall $\Sigma_\epsilon = \Sigma \cup \{\epsilon\}$)

Then **P accepts** w if there are

$r_0, r_1, \dots, r_n \in Q$ and

$s_0, s_1, \dots, s_n \in \Gamma^*$ (sequence of stacks) such that

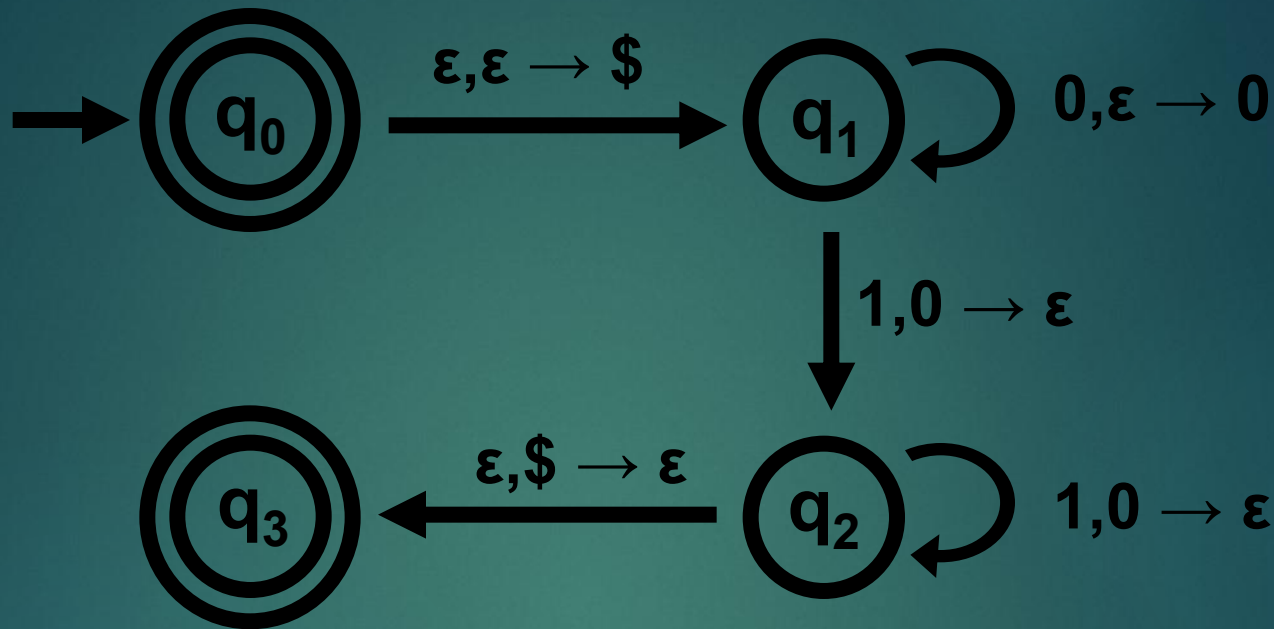
1. $r_0 = q_0$ and $s_0 = \epsilon$ (**P** starts in q_0 with empty stack)

2. For $i = 0, \dots, n-1$:

$(r_{i+1}, b) \in \delta(r_i, w_{i+1}, a)$, where $s_i = at$ and $s_{i+1} = bt$ for some $a, b \in \Gamma_\epsilon$ and $t \in \Gamma^*$

(**P** moves correctly according to state, stack and symbol read)

3. $r_n \in F$ (**P** is in an accept state at the end of its input)



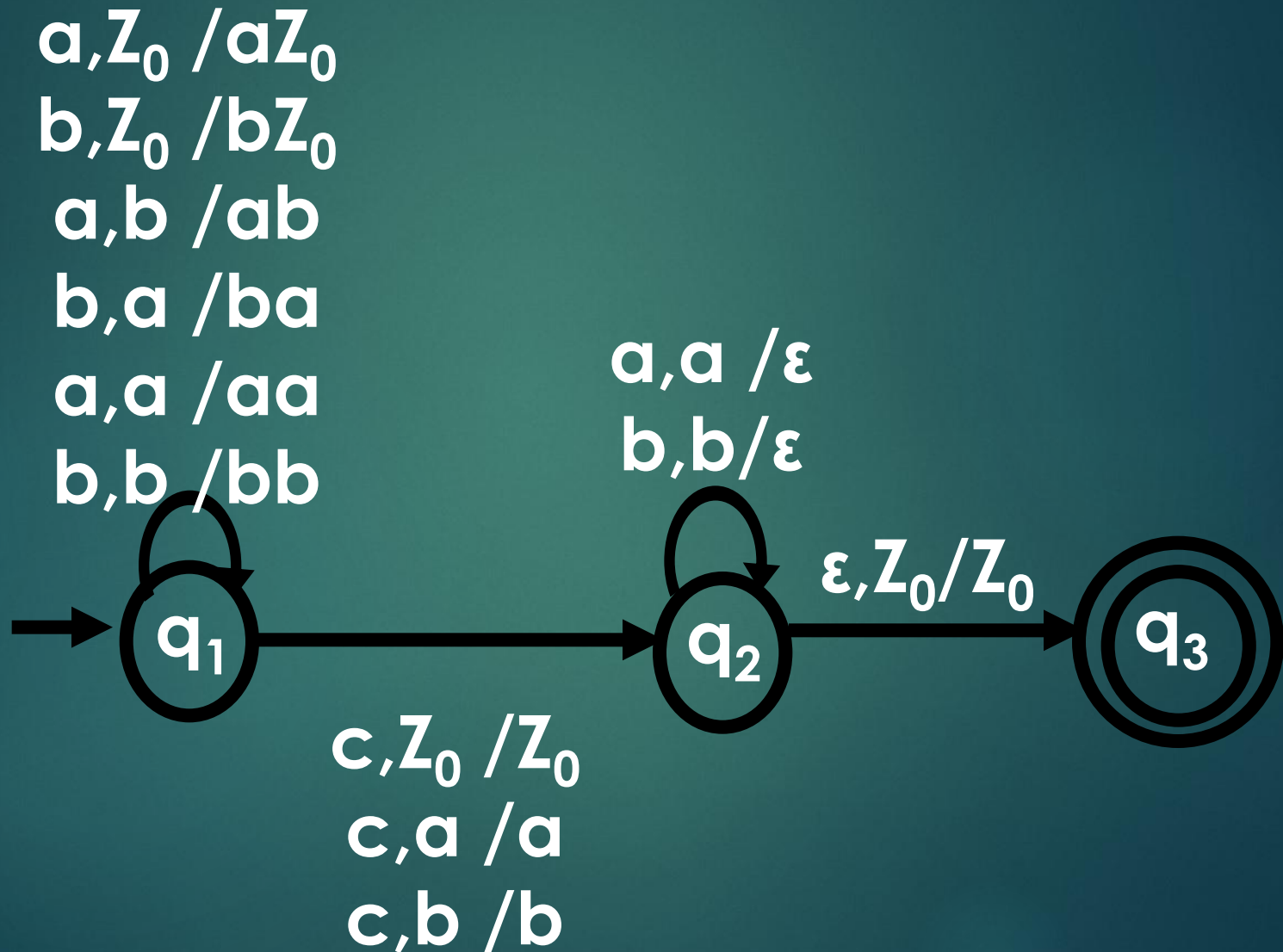
$$Q = \{q_0, q_1, q_2, q_3\} \quad \Sigma = \{0, 1\} \quad \Gamma = \{\$, 0, 1\}$$

$$\delta : Q \times (\Sigma \cup \epsilon) \times \Gamma \rightarrow Q \times \Gamma^*$$

$$\delta(q_1, 1, 0) = \{ (q_2, \epsilon) \} \quad \delta(q_2, 1, 1) = \emptyset$$

DETERMINISTIC PDA

ODD-LENGTH PALINDROMES $L = w c w^R$



NON DETERMINISTIC PDA

EVEN-LENGTH PALINDROMES $L = ww^R$

$a, Z_0 / aZ_0$

$b, Z_0 / bZ_0$

$a, b / ab$

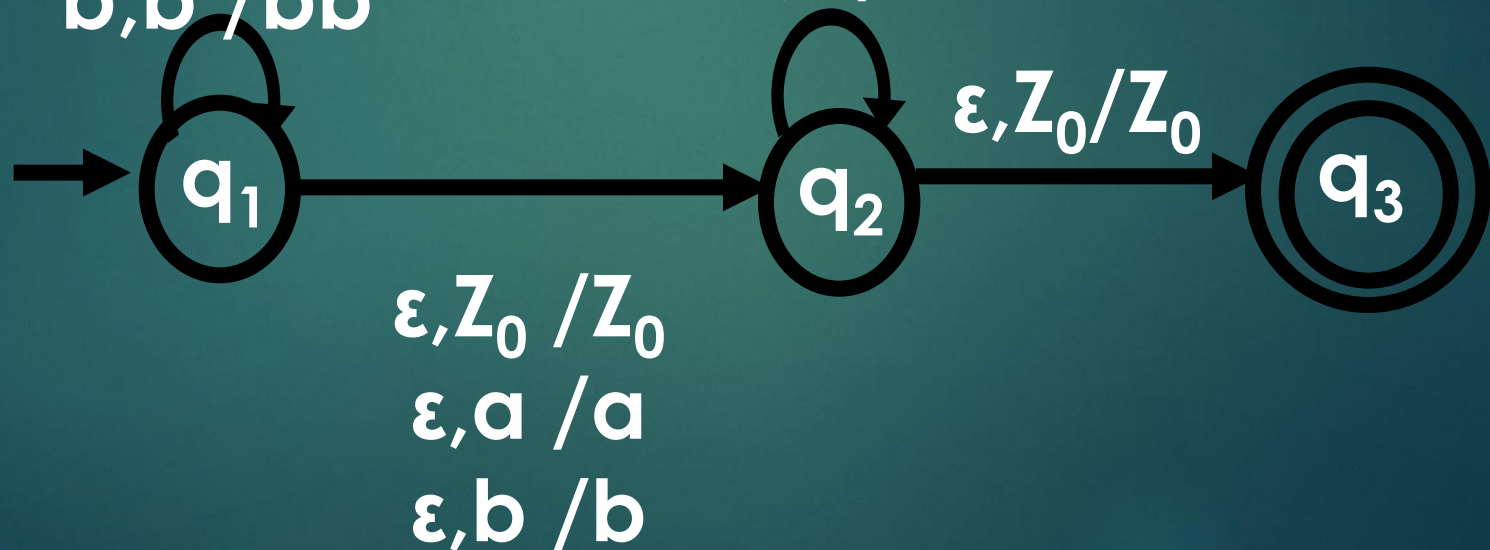
$b, a / ba$

$a, a / aa$

$b, b / bb$

$a, a / \epsilon$

$b, b / \epsilon$



Instantaneous Descriptions

- ▶ The ID must record the state and stack contents:
- ▶ If $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ be a PDA then

$(q, aw, za) \vdash (p, w, \beta a)$

If $\delta(q, a, z) = (p, \beta)$

Consumes **a** from the input

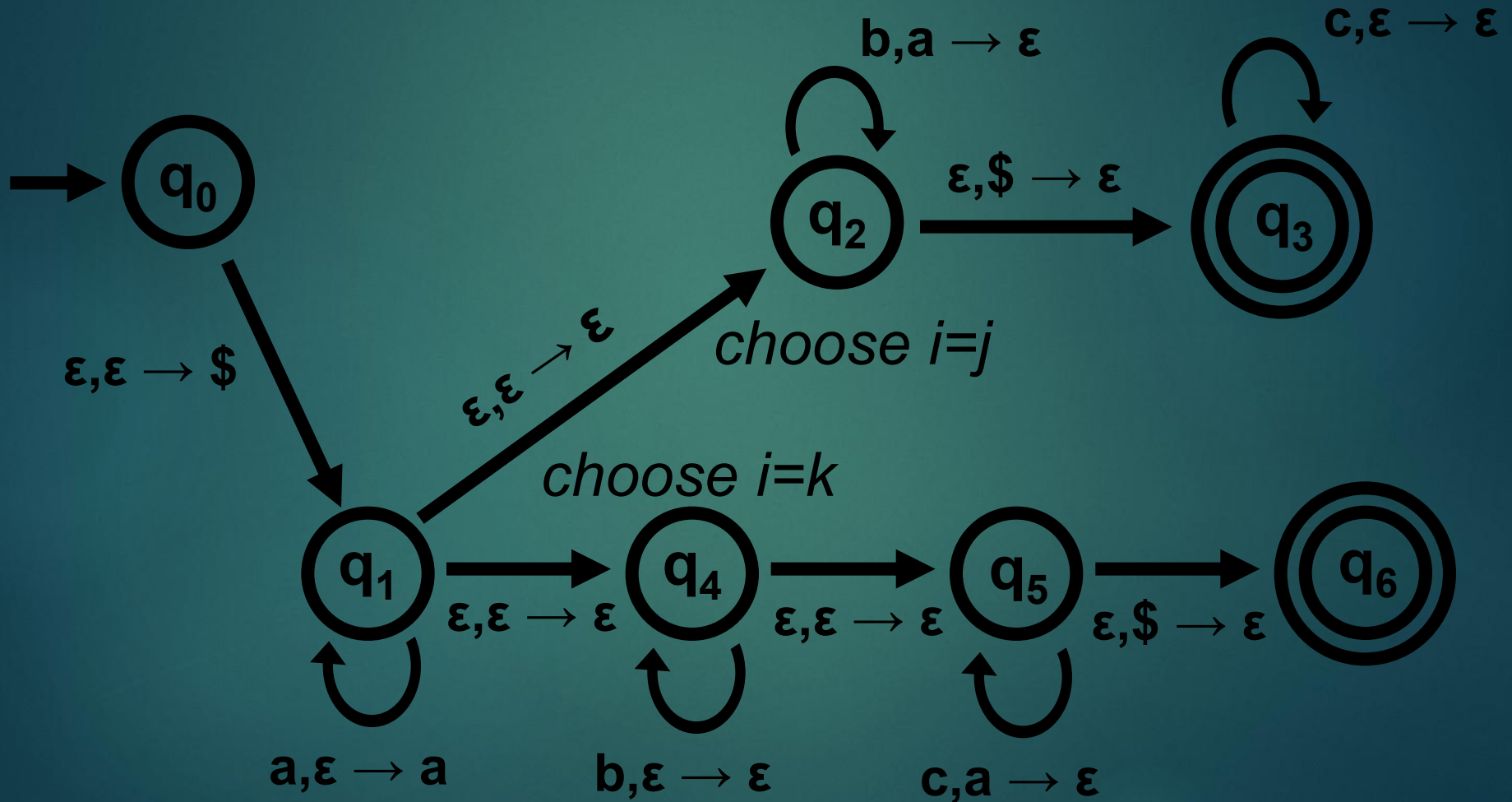
replace **z** on top of the stack by **β**

Go from state **q** to **p**

Assignment-1

1. Theorem 2.12 – includes 2.11-63,64 (8 marks)
2. Theorem 2.22 – page no. 79-(8 marks)
3. Theorem 3.7 –page no. 102,103 (8 marks)
4. Explain closure properties of RL-16 marks
5. Theorem 6.9 -page no. 231 (8 marks)
6. Theorem 6.11 -page no. 235 (8 marks)

Build a PDA to recognize
 $L = \{ a^i b^j c^k \mid i, j, k \geq 0 \text{ and } (i = j \text{ or } i = k) \}$



Equivalence of PDA and CFG



1. CFL
2. Languages accepted by final state PDA
3. Languages accepted by empty stack PDA

Same

CFG to PDA

Let $G=(V,T,R,S)$ be a CFG.

Construct the PDA P that accepts $L(G)$ by empty stack as follows:

$$P=(\{q\},T,V\cup T, \delta, q, \emptyset)$$

Where δ is defined by

For each variable A

$$R1: \delta(q,\varepsilon,A)=\{(q,\beta) \mid A\rightarrow\beta \text{ is a production in } G\}$$

For every terminal symbol a

$$R2: \delta(q,a,a)=\{(q, \varepsilon)\}$$

Convert the grammar CFG TO a PDA

$E \rightarrow E + E$

$E \rightarrow id$

► The equivalent PDA is given by:

► $P = (\{q\}, \{+, id\}, \{E, +, id\}, \delta, q, \emptyset)$

Where δ is defined by:

$\delta(q, \varepsilon, E) = \{(q, E + E), (q, id)\}$

$\delta(q, id, id) = \{(q, \varepsilon)\}$

$\delta(q, +, +) = \{(q, \varepsilon)\}$

Convert the grammar CFG TO a PDA

$$S \rightarrow 0S1 \mid A$$
$$A \rightarrow 1A0 \mid S \mid \varepsilon$$

► The equivalent PDA is given by:

► $P = (\{q\}, \{0, 1\}, \{S, A, 0, 1\}, \delta, q, S)$

Where δ is defined by:

$$\delta(q, \varepsilon, S) = \{(q, 0S1), (q, A)\}$$
$$\delta(q, \varepsilon, A) = \{(q, 1A0), (q, S), (q, \varepsilon)\}$$
$$\delta(q, 0, 0) = \{(q, \varepsilon)\}$$
$$\delta(q, 1, 1) = \{(q, \varepsilon)\}$$

Assignment

Construct a PDA from the grammar CFG

1. $S \rightarrow 0BB \mid A$
 $B \rightarrow 0S \mid 1S \mid 0$
2. $E \rightarrow E+E \mid E^*E \mid (E) \mid I$
 $I \rightarrow a \mid b \mid Ia \mid Ib \mid I0 \mid I1$
3. $E \rightarrow E+T \mid T$
 $T \rightarrow T^*F \mid F$
 $F \rightarrow (E) \mid I$
 $I \rightarrow a \mid b \mid Ia \mid Ib \mid I0 \mid I1$

PDA to Grammar

$P = (Q, T, \Gamma, \delta, q_0, Z_0, \epsilon)$. Construct $G = (V, T, P, S)$

Where V consists of

$V = \{S\} \cup \{ [q, Z, q'] \mid q, q' \text{ belongs to } Q, Z \text{ belongs to } \Gamma. \}$

The productions of G are:

R1: $S \rightarrow [q_0, Z_0, q]$ for every q in Q .

R2: Each move erasing a pushdown symbol given by $\delta(q, a, Z) = (q', \epsilon)$ induces the production $[q, Z, q'] \rightarrow a$.

R3: Each move not erasing a pushdown symbol given by $\delta(q, a, Z) = (q_1, Z_1 Z_2 Z_3 \dots Z_m)$ induces many productions of the form

$[q, Z, q'] \rightarrow a [q_1, Z_1, q_2] [q_2, Z_2, q_3] \dots [q_m, Z_m, q']$

Where each of the states $q', q_2, \dots$



Then for all list of states $r_1 r_2 r_3 \dots r_k$

G has the production

$$[qXr_k] \rightarrow a[ry_1r_1][r_1y_2r_2]\dots[r_{k-1}y_kr_k]$$

This production says that one way to pop X and go from state q to state r_k is to read a then use some input to pop y_1 off the stack while going from state r to state r_1 , then read some more input that pops y_1 off the stack goes from state r_1 to state r_2 and so on.

1. Construct Grammar from PDA

Given PDA: $(\{p, q\}, \{0, 1\}, \{X, Z_0\}, \delta, q, Z_0)$ to CFG

1. $\delta(q, 1, Z_0) = \{(q, X Z_0)\}$

2. $\delta(q, 1, X) = \{(q, XX)\}$

3. $\delta(q, 0, X) = \{(p, X)\}$

4. $\delta(q, \epsilon, X) = \{(q, \epsilon)\}$

5. $\delta(p, 1, X) = \{(p, \epsilon)\}$

6. $\delta(p, 0, Z_0) = \{(q, Z_0)\}$

The variables involved in Grammar are:

- ▶ Number of states=2
- ▶ Number of stack symbols=2

$$S0, \text{ Total Number of variables} = 1 + (2 \times 2 \times 2) \\ = 9$$

$$V = \{S, [qXq], [qXp], [pXq], [pXp], [qZ_0q], [qZ_0p], [pZ_0q], [pZ_0p]\}$$

The Productions involved in Grammar are:

- ▶ Using First rule

- ▶ $S \rightarrow [qZ_0 q]$

- ▶ $S \rightarrow [qZ_0 p]$

(or)

- ▶ $S \rightarrow [qZ_0 q] / [qZ_0 p]$

Using Second rule and first transition function

► $\delta(q, 1, Z_0) = \{(q, X Z_0)\}$

$[q Z_0 q] \rightarrow 1[qXq] [q Z_0 q] / 1[qXp][p Z_0 q]$

$[q Z_0 p] \rightarrow 1[qXq] [q Z_0 p] / 1[qXp][p Z_0 p]$

$\delta(q, 1, X) = \{(q, XX)\}$

$[q X q] \rightarrow 1[qXq] [q X q] / 1[qXp][p X q]$

$[q X p] \rightarrow 1[qXq] [q X p] / 1[qXp][p X p]$

$$\delta(q, 0, X) = \{(p, X)\}$$

$$[q \ X \ q] \rightarrow 0[p \ X \ q]$$

$$[q \ X \ p] \rightarrow 0[p \ X \ p]$$

$$4. \ \delta(q, \varepsilon, X) = \{(q, \varepsilon)\}$$

$$[q \ X \ q] \rightarrow \varepsilon$$

$$5. \ \delta(p, 1, X) = \{(p, \varepsilon)\}$$

$$[p \ X \ p] \rightarrow 1$$

$$6. \ \delta(p, 0, Z_0) = \{(q, Z_0)\}$$

$$[p \ Z_0 \ q] \rightarrow 0[q \ Z_0 \ q]$$

$$[p \ Z_0 \ p] \rightarrow 0[q \ Z_0 \ p]$$

2. Construct Grammar from

Given PDA: $(\{q_0, q_1\}, \{0, 1\}, \{X, Z_0\}, \delta, q_0, Z_0)$ to CFG

1. $\delta(q_0, 0, Z_0) = \{(q_0, XZ_0)\}$

2. $\delta(q_1, 1, X) = \{(q_1, \epsilon)\}$

3. $\delta(q_0, 0, X) = \{(q_0, XX)\}$

4. $\delta(q_1, \epsilon, X) = \{(q_1, \epsilon)\}$

5. $\delta(q_0, 1, X) = \{(q_1, \epsilon)\}$

6. $\delta(q_1, \epsilon, Z_0) = \{(q_1, \epsilon)\}$

The variables involved in Grammar are:

- ▶ Number of states=2
- ▶ Number of stack symbols=2

$$S0, \text{ Total Number of variables} = 1 + (2 \times 2 \times 2) \\ = 9$$

$$V = \{S, [qXq], [qXp], [pXq], [pXp], [qZ_0q], [qZ_0p], [pZ_0q], [pZ_0p]\}$$

$$T = \{0, 1\}$$

The variables involved in Grammar are:

- ▶ Number of states=2
- ▶ Number of stack symbols=2

$$S0, \text{ Total Number of variables} = 1 + (2 \times 2 \times 2) \\ = 9$$

$$V = \{S, [q_0Xq_0], [q_0Xq_1], [q_1Xq_0], [q_1Xq_1], [q_0Z_0q_0], [q_0Z_0q_1], [q_1Z_0q_0], [q_1Z_0q_1]\}$$

$$T = \{0, 1\}$$

Grammar

▶ $S \rightarrow [q_0 Z_0 q_0] / [q_0 Z_0 q_1]$

$[q_0 Z_0 q_0] \rightarrow 0[q_0 X q_0][q_0 Z_0 q_0] / 0[q_0 X q_1][q_1 Z_0 q_0]$

$[q_0 Z_0 q_1] \rightarrow 0[q_0 X q_0][q_0 Z_0 q_1] / 0[q_0 X q_1][q_1 Z_0 q_1]$

▶ $[q_1 X q_1] \rightarrow 1$

▶ $[q_0 X q_0] \rightarrow 0[q_0 X q_0][q_0 X q_0] / 0[q_0 X q_1][q_1 X q_0]$

▶ $[q_0 X q_1] \rightarrow 0[q_0 X q_0][q_0 X q_1] / 0[q_0 X q_1][q_1 X q_1]$

▶ $[q_1 X q_1] \rightarrow \epsilon$

▶ $[q_0 X q_1] \rightarrow \mathbf{1}$

▶ $[q_1 Z_0 q_1] \rightarrow \epsilon$

3. Convert PDA to Grammar

- ▶ $P_N P = (\{q\}, \{i, e\}, \{Z\}, \delta_N, q, Z)$
- ▶ 1. $\delta_N(q, i, Z) = (\{q, ZZ\})$
- ▶ 2. $\delta_N(q, e, Z) = (\{q, \epsilon\})$

Number of states=1

Number of stack symbols=1

S0, Total Number of variables = $1 + (1 \times 1 \times 1)$
=2

$V = \{S, [qZq]$

$T = \{i, e\}$

Grammar

- ▶ $S \rightarrow [qZq]$
- ▶ $\delta_N(q, i, Z) = (\{q, ZZ\})$
 $[qZq] \rightarrow i[qZq][qZq]$
- ▶ $\delta_N(q, e, Z) = (\{q, \varepsilon\})$
 $[qZq] \rightarrow e$

RENAME - $[qZq]$ is renamed to A

$S \rightarrow A$

$A \rightarrow iAA \mid e$

(since A and S derive same strings)

$G = (\{S\}, \{i, e\}, S \rightarrow iSS \mid e, S)$

4. Construct Grammar from

Given PDA: $(\{q, p\}, \{0, 1\}, \{Z_0, X\}, \delta, q, Z_0, \{p\})$ to CFG

1. $\delta(q, 0, Z_0) = \{(q, X Z_0)\}$

2. $\delta(q, 0, X) = \{(q, XX)\}$

3. $\delta(q, 1, X) = \{(q, X)\}$

4. $\delta(q, \epsilon, X) = \{(p, \epsilon)\}$

5. $\delta(p, \epsilon, X) = \{(p, \epsilon)\}$

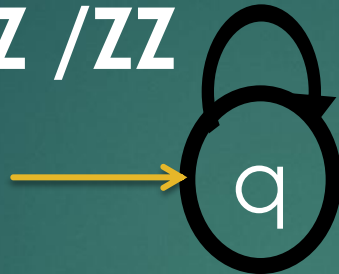
6. $\delta(p, 1, X) = \{(p, XX)\}$

7. $\delta(p, 1, Z_0) = \{(p, \epsilon)\}$

Empty stack PDA to Final State PDA

$e, Z / \epsilon$

$i, Z / ZZ$

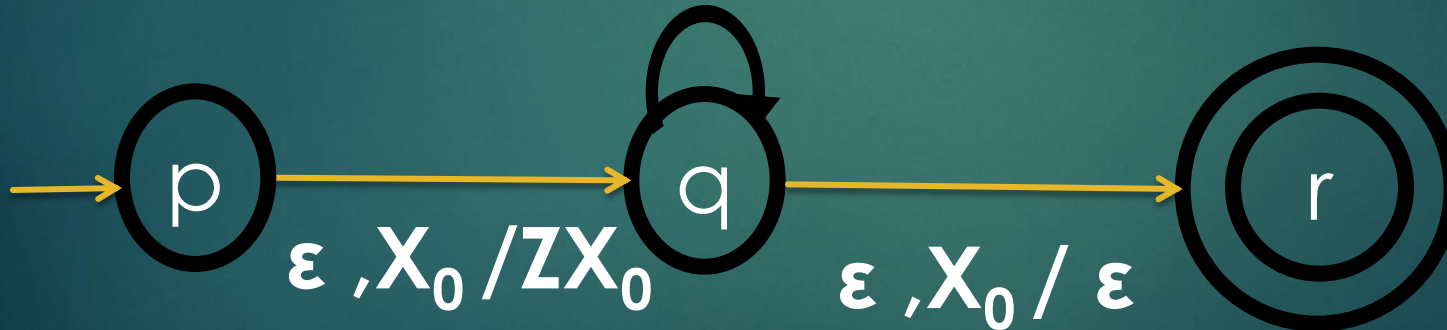


Empty stack PDA

$e, Z / \epsilon$

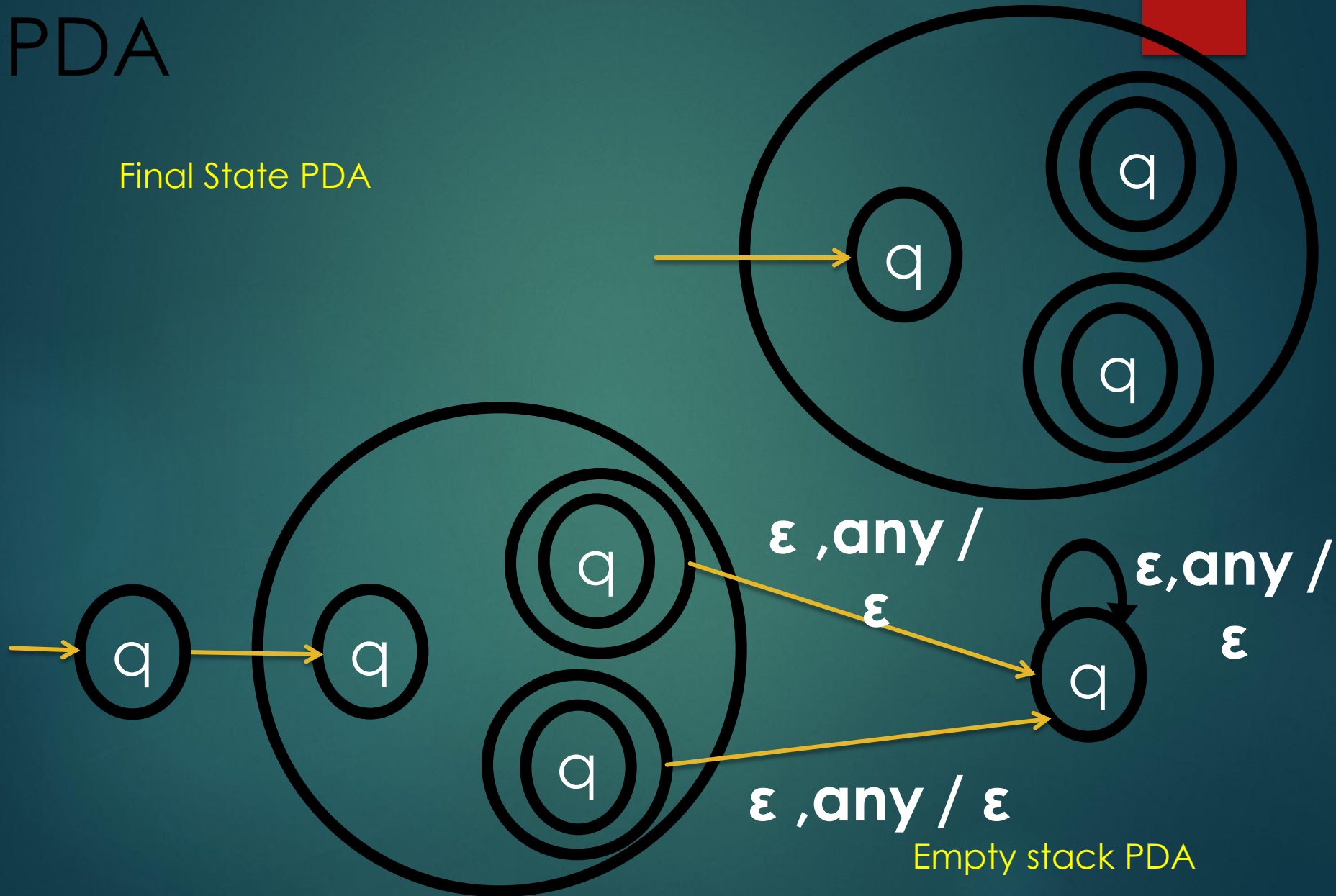
$i, Z / ZZ$

Final state PDA



Final State PDA to Empty stack PDA

Final State PDA



Summary

- CFG and CFL
- Derivations : LMD, RMD, Parse Tree
- Ambiguous Grammar .
- Push Down Automata (PDA)
- Instantaneous Description - Language of PDA:
- Acceptance by Final state, Acceptance by empty stack
- Deterministic PDA
- Equivalence of PDA and CFG
 - CFG to PDA
 - PDA to CFG